Four Dark Corners of Object Protocols

Anonymous

Abstract

An object protocol is a set of constraints on how objects can be used. Different techniques have been proposed to specify object protocols and verify that they are respected. We identified a number of practical problems with object protocol specification that relate to four important concepts of object-oriented programming. Object protocols aim to model method sequences but must handle object state; They must handle unpredictable modifications by subtypes; They are a function of an object's state but overlook extrinsic state; They target types but can be instance-specific. We submit that the case for specifying object protocols as a standalone effective way to improve code quality is not made, and that their role may be best found in complementing pre- and post-condition specifications.

2012 ACM Subject Classification Software and its engineering → Object oriented languages; Software and its engineering → Designing software; Software and its engineering → Software verification and validation

Keywords and phrases Object protocol, typestate, object usage, API protocol, state machine

Category Pearls

1 Introduction

Programming with objects requires respecting constraints on how particular objects can be used. For over 30 years, the term *object protocol* has been employed to refer to such constraints [14]. Over the years, different techniques have been proposed to specify object protocols, based on formalisms that include regular expressions [5], state machines [10], and typestate [3]. Techniques have also been proposed to infer object protocol specifications from existing systems (e.g., [12, 15]).

Despite their potential for improving code quality, object protocol specifications are still not a part of mainstream programming. Why? In an attempt to answer this question, we revisited the concept from the perspective of contemporary object-oriented programming practices and thereby identified practical problems with object protocol specifications that relate to four important concepts of object-oriented programming. These four *dark corners* are manifestations that a system to specify legal usage patterns for objects cannot handle the richness and diversity of common programming idioms and practices in evidence today.

In the light of these dark corners of object protocols, we submit that specifying pre-and post-conditions offers a more compelling value proposition for expressing usage constraints on objects, from which approximate object protocol specifications can be extracted as a derived artifact to guide the development of client code.

2 The Dark Corners

We discuss how four foundational concepts of object-oriented programming conflict with the goal of specifying object protocols. We enrich the discussion with samples from the Java 25 Class Library. Quotes without attribution in the discussion of code samples originate from the reference documentation of the corresponding types [11].

2.1 Tension between Sequences and States

The literature on object protocols betrays a clear preoccupation with capturing legal *sequences* of method calls: *"It often happens than one operation on an object must precede some other..."* [14]; *"often there are rules constraining the order in which messages may be sent"* [16]; *"sequencing constraints on the order in which methods may be called"* [5]; *"frequently the specification also defines a protocol of allowed method call sequences"* [3], etc. The canonical example is of a file handler object that requires the file to be open before read or write operations can occur, and closed afterwards. In essence, however, this constraint amounts to a precondition on some of the methods, which could include the object's constructor or finalizer. At the same time, there are formidable obstacles to expressing such constraints as method sequences. Reasoning in terms of method sequences implies accounting for the *implicit state* induced by the side effect of method calls. As an example, the trivial precondition that a method pop should not be called on an empty Stack cannot be readily expressed as a regular language. Indeed, precisely capturing the constraints on sequences of method calls requires an expressive specification language (e.g., pushdown automata) that may require incorporating some of the complexities of the implementation.

In practice, object protocols can be exclusively concerned with the state of an object, such that method sequences do not matter in the least. For example, for an Iterator, the only thing that matters when calling next is that there is at least one more element to iterate over. The fact that a method hasNext is usually called before next is an empirical reality but not a requirement, and modeling the number of potential calls to next is equivalent to modeling the number of elements to iterate over, a piece of information purposefully hidden by the design of the Iterator.

> **java.util.Iterator**: This interface allows iterating over the elements of an aggregate object. It declares methods next and hasNext such that next is only enabled if hasNext would return true. Iterator also declares a method remove that can only be called at most once after each call to next, with the effect of removing the previously-traversed element from the underlying collection. Interface Iterator also introduces a so-called *multi-object protocol* [12] in that the behavior of the iterator *"is unspecified if the underlying collection is modified while the iteration is in progress in any way other than by calling [remove]"*. To add to the challenge of reasoning about protocols for Iterator, method remove is documented as *optional*, meaning that implementing types can legitimately throw an exception instead of supplying an implementation for it.

Reasoning about object state is thus impossible to avoid in the specification of object protocols and a number of approaches account for this realization explicitly, starting with Nierstrasz's proposal using state machines [10]. The related concept of *typestate* [13] also seems, at first glance, to be a natural fit for the problem of specifying object protocols. Introduced in a non-object-oriented context, the goal of *typestate tracking* is *"detecting at compile-time syntactically legal but semantically undefined execution sequences"* [13], such as accessing uninitialized variables. Typestate tracking was applied to object-oriented programming roughly two decades after its proposal [3]. Notwithstanding the relation to state implied by the term *typestate*, the resulting states have no formal relation to the concrete states of an object and can be considered, in fact, accidental specification elements aimed at defining languages, thus failing to break free of the lure of modeling sequences.

Why attempt to model transitions between abstract states as opposed to the required object state to meet a precondition upon execution of a method? In the case of our file

handler, what matters if we want to read a file is not so much that open was called, but that *some program execution event* (method call, field assignment, object initialization) left the program in a state that meets the precondition for calling read.

An important distinction between object protocols and preconditions is one of scope: preconditions apply to individual methods whereas, in principle, an object protocol models object usage comprehensively. This distinction can be either a benefit or a drawback, depending on the perspective. Reasoning about preconditions favors an on-demand and modular approach to object behavioral conformance. Yet, reasoning about each precondition in isolation can hinder understanding how the methods of a class cooperate to realize a service. Some questions programmers may have could involve reasoning about the universe of allowable sequences of valid (i.e., precondition-satisfying) method calls. For example, in the case of our iterator, is is possible to call remove twice in a row to remove two consecutive elements? Such questions are naturally formulated and answered in terms of sequences.

Synthesis: Although specifications of object protocols favor abstracting object state and focusing on method sequences as the primary concern, they can only approximate protocol constraints. Modeling object state plays a crucial role in expressing object usage constraints precisely. Pre- and post-condition specifications have all the necessary information to capture object protocols precisely. Their modularity makes them a convenient source for answering many developer questions but hampers understanding method cooperation. Approximate sequence based specification derived from pre- and post-conditions (e.g., as proposed by de Caso et al. [6]) could mitigate this drawback.

2.2 Tension with Polymorphism

Constraints on the use of types must be expressed in terms of the interface they define, and proposals for capturing object protocols extend abstract specifications [3, 5, 16]. The underlying philosophy is that *"errors made in protocol design are detected at a higher level of abstraction, independent of the program's source code"* [9]. This postulate introduces the problem of specifying object behavior in terms of abstract states without knowledge of their concrete state space, and thus of the partitions of this concrete space that would form the abstract state space. This challenge is illustrated by the Iterator interface, described above.

Although considered an object protocol [2], the only method with an expected side effect *on the iterator itself* is next, and therefore any protocol specification on an iterator consists of a sequence of zero or more calls to next, until is it no longer possible to call it. In this case, the relevant abstract state space is determined by the simple Boolean enablement condition for next determined by hasNext [8], and a protocol provides limited benefits over the corresponding precondition. However, client code may need to be aware of additional conditions determined by subtypes, both abstract (i.e., subinterfaces) and concrete (i.e., implementing types). Both abstract and concrete subtypes introduce additional complexities for reasoning about usage constraints on objects. Starting with abstract subtypes, previous work has concerned itself with the impact of a protocol specification on subtypes, but this effort focused on establishing the conformance of subtypes to supertypes through specifications [5, 3]. In practice, subtypes can have unpredictable impacts on a supertype's protocol.

java.util.ListIterator: The behavior defined by Iterator is extended by interface ListIterator, which supports traversing the underlying structure in either direction (adding previous to next). The ListIterator also supports modifying the underlying structure through methods such as add and set. These methods introduce their own additional protocols. For instance, set can only be called *"if neither remove nor add have been called after the last call to next or previous"*. The specification for ListIterator also redefines the protocol of remove to account for traversal in reverse order.

With this example of interface extension, we observe how protocols can be almost arbitrarily modified through subtyping. While it may be possible to capture some of the extensions with additional formalism, such as state refinement [3], we cannot get around the fundamental limitation that we are trying to model abstract state while being ignorant, by design, of the corresponding concrete state. For instance, a concrete Iterator could declare a method reset to return the iterator to its initial abstract state, and thus enrich its protocol with an additional transition. It is also possible (if ill-advised) to implement a circular iterator whose hasNext method never returns false, hence eliminating the abstract state that disables method next. Specifying protocols in terms of method sequences also assumes an ideal application of object-oriented programming with complete respect of encapsulation principles. In practice, a flawed implementation of a Stack (such as java.util.Stack), could enable arbitrary modifications to the stack that bypass a push operation, making the implicit state of the object that much more difficult to track. In the same vein, the non-implementation of optional methods by subtypes converts one protocol ("only call remove after next") to a different one ("don't call remove"). Ultimately, the only sure way to detect and understand a protocol may be to analyze the implementation of the corresponding type. Indeed, in their study of object protocols in the wild, Beckman et al. only recognize an object protocol if there is an implementation that verifies it [2].

Inheritance also has noteworthy repercussions on object protocols. In previous work that specifically targets inheritance, the focus was on ensuring that *"the behavior specified in the usage protocol of the superclass is also possible in its subclasses"* [1]. In practice, however, protocols are reflected in an implementation, whether they are specified or not, and subclasses inherit this potentially tacit protocol. A pervasive example is that all Java exceptions inherit the protocol of Throwable, although they may deactivate it and thus become non-substitutable. Different subclasses can also implement a protocol inconsistently, thus requiring vague documentation of potential constraints in the supertype.

java.lang.Throwable: Any exception object in Java is of a subtype of Throwable. An exception can have a *cause*, which is another exception. An exception's cause is stored in a field cause of class Throwable. The cause of an exception can be set via the constructor of Throwable or through a call to method initCause. It is not possible to set the cause of an exception twice. This means that it is not possible to call initCause more than once. It is also possible to set the value of field cause through the constructor (which does not call initCause).

Synthesis: By extending its supertype, a subtype can add or remove allowable sequences of method calls. In practice, modification of an object protocol by subtypes can occur even when the subtype conforms to the Liskov Substitution Principle: a subtype method with a weaker precondition may support additional method sequences, while one with a stronger postcondition may restrict future interactions with the object.

2.3 Tension between Intrinsic and Extrinsic State

Specifying the validity of method sequences requires modeling of object state on some level. The canonical examples discussed in the literature involve state machines based on state that is inherent to an object, either implicitly or explicitly, e.g., a stream is open or closed, a stack is empty or not, etc. However, these simple predicates do not expose how challenging it can be to model abstract object state. A particularly pernicious question concerns what to do with *extrinsic state*.

In it simplest expression, an object consists of instance variables that capture attributes that are an essential representation of its nature: its *intrinsic state*. In a course management system, attributes such as studentId or name would be part of the intrinsic nature of a Student. But let us assume a badly-designed system with circular dependencies, wherein a Student object also includes an instance variable courses, that collects the courses the student is enrolled in. To work with a maximally disturbing scenario, we will say that a course aggregates all Students enrolled in it. Technically, the state of a Student object will include all reachable Student objects, and thus possibly the entire state of the running program. It is not hard to envision how we would like to specify usage constraints over such a design, as terrible as it is. For example, a method enroll could be disabled if a course is full, etc. How can we reason about object state under those conditions? The concept of *extrinsic state*, possibly introduced in a discussion of the FLYWEIGHT pattern by Gamma et al. [7], can help us provide an answer: designers can determine, through documentation, annotations, or API design, what part of an object's state is intrinsic and what part is extrinsic. Hypothetically, an object-oriented program could be sufficiently annotated to indicate which state is extrinsic, and omit such state from the protocol specification.

Unfortunately, it can sometimes be very difficult to decide whether some state is extrinsic or intrinsic. With circular dependencies, as in the facetious example above, we could "cut" at the first opportunity. But many patterns and idiosyncratic designs are not so easily handled. A particularly challenging issue is caused by wrapper objects. A canonical example is provided by the COMMAND design pattern (for example for use in the menubar of a drawing end-user application). A *command* object will typically contain a reference to the context in which the command will be executed. This context could be the entire application context (the drawing application), or a subset of its state (e.g., some sort of feature). In this case, we could say that properties of the command (e.g., the name, the number of time it was executed) is the intrinsic state and the context for executing the command is the extrinsic state. While the boundary is somewhat clear, the issue is that a protocol for disabling the command will most likely need to be expressed in terms of the extrinsic state. Decorator objects suffer from a similar limitation but exhibit a different symptom: the protocol for properly using a decorator may depend on the decorated object. While this issue would seem to violate the principles of information hiding, the reality is that such cases exist. Finally, whether to provide the extrinsic state necessary for a method to fulfill its mandate through arguments or an instance variable can be, in many cases, an arbitrary design decision. Yet, some treatments of object protocols disregard parameters and thus possibly extrinsic state (e.g., [2]).

> **java.io.OutputStream:** This abstract class *"is the superclass of all classes representing an output stream of bytes"*. It declares, among others, a method close and a method write with three overloaded variants with an inconsistent protocol related to close. ▷

The documentation for one variant indicates that an exception *is* thrown if the stream is closed, but another mentions that an exception *may be thrown*. In any case, different subclasses implement different protocols: ByteArrayOutputStream tolerates writing after closing, but FileOutputStream does not. In some cases, the protocol can be based on composition: FilterOutputStream is a decorator that wraps another output stream and its protocol for write depends on the wrapped stream. Finally, some subclasses of OutputStream are private types returned by factory methods (e.g., Base64.getEncoder().wrap(o)), and thus their protocol can only be discovered by inspecting the source code, if available.

Synthesis: An object's state comprises an intrinsic and an extrinsic component. Definitions of object protocols as constraints over sequences of method calls require a precise distinction between values that are part of the state of an object for the purpose of protocol definition and those that are out of bounds. Deciding between the two can be a subjective exercise.

2.4 Tension between Types and Instances

Object protocols are usually understood as specifications, to be attached to types. However, even in strongly-typed programming languages, cases can arise where one must reason about object protocols as being attached to particular instances. Three mechanisms in particular favor this kind of object-centric usage constraints: constructors, immutable objects, and factory methods.

A *constructor* can be designed in such a way as to instantiate an object so that their protocol is altered, typically by permanently disabling some methods. For example, the cause of a Throwable (see above) can only be set once. Thus, instantiating a Throwable or any of its subtypes with a constructor that sets the cause permanently disables the initCause method for the resulting objects. In cases where the object construction dictates a protocol different from that of the superclass, then we are in the presence of an issue with inheritance, as discussed in Section 2.2. For example, an exception type with a single constructor that calls a superconstructor that initializes the cause will disable initCause for *all* objects of the class. However, constructor overloading could allow the instantiation of exception types with two different protocols. In terms of state machine models, the two variants can be considered instances of the same protocol: instantiating exceptions with a cause simply initializes the object in the abstract state in which the initCause method is disabled. In practice, however, these objects are initialized differently in client code and awareness of this difference is an important benefit of specifying protocols.

Immutable objects can have different protocols depending on the constant values that they store. A common issue is that some methods may not be applicable to the state captured by the object, such as for instances of BigInteger.

java.math.BigInteger: This class represents integers of arbitrary precision. In contrast to the wrapper type Integer, which supports most operations through static methods, operations on BigInteger are implemented as instance methods. Not all methods are applicable to all implicit arguments. For example, nextProbablePrime and sqrt throw an exception for negative numbers, and any method dealing with prime numbers is disabled for excessively large values.

That arithmetic functions fail on inapplicable arguments is by no means shocking. However, when such functions are implemented as instance methods, their preconditions match most definitions of an object protocol, which raises the question of whether an exception should be considered for immutable objects. We are not aware of any attempt to model object protocols that considered this eventuality.

Finally, *factory methods* are methods designed to create objects while hiding the concrete type of the object created. Although hailed as a benefit [4, Item 1], this design technique produces methods that return objects whose concrete type can be unknown and/or private. This last argument relates to the type of the object, as opposed to its value, and thus its implications dovetail with the issues we highlighted as a tension with polymorphism (see Section 2.2). However, the fact that the type of objects returned by factory methods is statically unknown means they can alter their supertype's protocol in unpredictable or unintuitive ways, and thus introduce the requirement to reason about object usage constraints in terms of specific instances, as in the case of dynamically-typed languages.

> **java.util.List:** This interface is the supertype of any implementation of the *list* abstract data type. The Java Class Library provides list implementations both through public types, such as ArrayList and LinkedList, and through factory methods such as List.of and Collections.singletonList. Both methods return an object of a private nested class. These classes have similar but different protocols regarding modifications to the resulting list. For instance, while lists returned by both methods have their add operation disabled, the list returned by List.of supports a set operation while the one returned by singletonList does not.

Synthesis: Object protocols are a static concept and their specification is attached to types. In principle, each sequence in the protocol should be achievable by some adequate sequence of method invocations. Yet, upon instantiation, the allowable sequence of methods may be a subset of those described by the protocol for the type. Indeed, the creation-time conditions for immutable objects and concrete private types can lead to instances whose object protocol specification is notably more general than the instance's actual protocol.

3 Conclusion

The vision for object protocols is that by making explicit constraints on how an object can be used, these constraints can be statically verified and dynamically monitored, thereby improving code quality. Through advocacy for pre- and postcondition specification and run-time validation, design by contract already provides a mechanism for encoding expectations about how objects can be used. However, preconditions express conditions based on the present concrete state of an object, whereas protocol specifications express approximate conditions based on a model of an abstract state space for an object. For example, a precondition may state that an object cannot be read if the object is somewhat "closed", and a sequence-based specification can prescribe that read must not be called after close. At first glance, these approaches look like comparable alternatives. However, in practice, specifying object protocols precisely is much more difficult than to express preconditions based on object state testable through interface methods.

We argued that four problems significantly add to the difficulty of reasoning about object protocols. A first one is that the formalism required to reason about method sequences is essentially a state machine, which introduces a dual mindset for thinking about object

usage. For example, with an iterator, although we must only call next if hasNext is true, the latter is a method without side effect and its use its optional, so we are back to reasoning about state. A second problem is that, although we would like objects to be substitutable in accordance with usual subtyping rules, the reality is that subtypes govern many modulations of protocols that could be impossible to anticipate in the supertype without introducing a circular dependency. Third, modeling abstract state can become an arbitrary subjective exercise in the face of object graphs with shared, or even circular, references. In other words, an object protocol may actually depend, transparently or not, on other objects referenced by the target object. Finally, design decisions related to object construction and initialization may introduce protocols that only apply to certain instances, with no correspondence with other objects of the same type. Providing a general and usable method to specify usable object protocols in this labyrinthine context may very well be insurmountable.

We conclude that the case for specifying object protocols as a standalone effective way to improve code quality is not made. Mechanisms already exist to specify any precondition on the queryable state of an object through assertion checking, and some programming environments provide additional support for the static verification of protocol-like properties in limited scopes. Object protocols specifications, however, can complement preconditions by documenting how methods should interact.

Acknowledgements

We anonymously thank our colleagues for their insights, comments and pointers which have enriched this discussion. The title of the paper is a nod to "Four dark corners of requirements engineering" by Zave and Jackson, *ACM Transactions on Software Engineering and Methodology*, 1997.

References

1 Lorenzo Bacchiani, Mario Bravetti, Marco Giunti, João Mota, and António Ravara. A Java typestate checker supporting inheritance. *Science of Computer Programming*, 221:102844, 9 2022. doi:10.1016/j.scico.2022.102844.

2 Nels E. Beckman, Duri Kim, and Jonathan Aldrich. An empirical study of object protocols in the wild. In Mira Mezini, editor, *Proceedings of the 25th European Conference on Object-Oriented Programming*, pages 2–26, 2011.

3 Kevin Bierhoff and Jonathan Aldrich. Lightweight object specification with typestates. In *Proceedings of the 10th European Software Engineering Conference held jointly with the 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, page 217–226, 9 2005. doi:10.1145/1081706.1081741.

4 Joshua Bloch. *Effective Java*. Addison-Wesley, 3rd edition, 2018.

5 Sergey Butkevich, Marco Renedo, Gerald Baumgartner, and Michal Young. Compiler and tool support for debugging object protocols. In *Proceedings of the 8th ACM SIGSOFT International Symposium on the Foundations of Software Engineering*, page 50–59, November 2000. URL: https://dl.acm.org/doi/10.1145/357474.355052, doi:10.1145/357474.355052.

6 Guido de Caso, Víctor A. Braberman, Diego Garbervetsky, and Sebastián Uchitel. Enabledness-based program abstractions for behavior validation. *ACM Transactions on Software Engineering and Methodology*, 22(3):25:1–25:46, 2013. doi:10.1145/2491509.2491519.

7 Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

8 Javier Godoy, Juan Pablo Galeotti, Diego Garbervetsky, and Sebastián Uchitel. Enabledness-based testing of object protocols. *ACM Transactions on Software Engineering and Methodology*, 30(2):12:1–12:36, 1 2021. doi:10.1145/3415153.

9 Madhu Gopinathan and Sriram K. Rajamani. Enforcing object protocols by combining static and runtime analysis. In *Proceedings of the 23rd ACM SIGPLAN conference on Object-Oriented Programming Systems Languages and Applications*, page 245–260, October 2008. doi:10.1145/1449764.1449784.

10 Oscar Nierstrasz. Regular types for active objects. In *Proceedings of the 8th Annual Conference on Object-Oriented Programming Systems, Languages, and Applications*, page 1–15, October 1993. URL: https://dl.acm.org/doi/10.1145/165854.167976, doi:10.1145/165854.167976.

11 Oracle. Java® platform, standard edition & Java development kit version 25 API specification, 2025. Verified 2025-10-27. URL: https://docs.oracle.com/en/java/javase/25/docs/api/.

12 Michael Pradel, Ciera Jaspan, Jonathan Aldrich, and Thomas R. Gross. Statically checking API protocol conformance with mined multi-object specifications. In *Proceedings of the 34th IEEE/ACM International Conference on Software Engineering*, page 925–935, 6 2012.

13 Robert E. Strom and Shaula Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, SE-12(1):157–171, 1986. doi:10.1109/TSE.1986.6312929.

14 Jan van den Bos and Chris Laffra. Procol: A concurrent object-oriented language with protocols delegation and constraints. *Acta Informatica*, 28(6):511–538, 6 1991. doi:10.1007/BF01463943.

15 Andrzej Wasylkowski, Andreas Zeller, and Christian Lindig. Detecting object usage anomalies. In *Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, page 35–44, 9 2007. doi:10.1145/1287624.1287632.

16 Daniel M. Yellin and Robert E. Strom. Interfaces, protocols, and the semi-automatic construction of software adaptors. In *Proceedings of the 9th Annual Conference on Object-oriented Programming Systems, Language, and Applications*, page 176–190, October 1994. doi:10.1145/191080.191111.